

Microservices Asynchronous Communication & Event-Driven Architecture

Introduction

Il problema ereditato

"Le operazioni di lunga durata non possono essere gestite con comunicazione sincrona, a causa di: aumento della latenza, ridotta disponibilità, servizi strettamente accoppiati, e risorse bloccate durante l'attesa."

"La best practice per superare questi problemi — soprattutto per i processi che non richiedono una risposta immediata — è passare a comunicazioni asincrone basate su messaggi."

Definizione: comunicazione asincrona

Quando un servizio (il producer) deve comunicare qualcosa a un altro servizio (il consumer), il producer invia un evento o un messaggio e NON attende una risposta immediata.

I punti chiave:

1. **Cosa viene inviato:** un **evento** (notifica di un accadimento significativo o cambio di stato) oppure un **messaggio** (dato destinato all'elaborazione)
2. ★ **Il producer non attende:** *"Invia l'evento o il messaggio, dopodiché è libero di continuare le proprie operazioni. Il thread di esecuzione non è bloccato."*
3. **Dove viene inviato:** a un componente intermedio, il **message broker** (RabbitMQ o Apache Kafka)
4. **Cosa fa il broker:** *"Trattiene in modo affidabile l'evento finché il servizio consumer non è pronto a elaborarlo."*
5. **Il protocollo:** AMQP (Advanced Message Queuing Protocol)

*"La comunicazione avviene tramite eventi o messaggi, e il producer fa **"fire and forget"** (invia e dimentica), il che disaccoppia fundamentalmente il producer dai consumer."*

Il ruolo del message broker / event bus

Il broker:

- **riceve** eventi/messaggi dai producer
- **li memorizza** — in modo **persistente** (se configurato) o **in memoria**
- **garantisce la consegna** ai consumer che si sono **sottoscritti** a quei tipi di evento
- ★ *"Se un consumer è temporaneamente inattivo o occupato, il broker può conservare gli eventi e ritentare la consegna in un secondo momento."*

★ Il concetto centrale: disaccoppiamento temporale

*"Per comunicare in modo asincrono, **non è necessario che producer e consumer siano direttamente collegati, né che siano operativi nello stesso momento. Questo disaccoppiamento temporale rappresenta un enorme vantaggio.**"*

*"Il broker attenua **il problema principale della comunicazione sincrona diretta: l'accoppiamento stretto e immediato.**"*

Ricorda la Lezione 8: il *temporal coupling* era uno dei 4 problemi della catena sincrona. **Il broker lo elimina alla radice.**

Asynchronous Communication Types: one-to-one / one-to-many

Ci sono **due** modelli, ed è importante non confonderli.

Tipo 1 — One-to-One (Point-to-Point), con code

Un producer invia un messaggio a una coda specifica. Tipicamente c'è UN servizio consumer in ascolto su quella coda.

Quando usarlo:

*"Ottimo quando si vuole **scaricare un'attività specifica o inviare un COMANDO** a un altro servizio per essere elaborato in modo asincrono, e ci si aspetta che solo un servizio gestisca quella specifica istanza di attività."*

Esempio:

*"Il microservizio **Basket** invia un comando "process order" alla coda, e **una singola istanza di Ordering** lo raccoglie ed elabora."*

Comportamento:

- *"Una volta che un messaggio è stato consumato ed elaborato con successo, **viene rimosso dalla coda**"*
- *"Se il consumer di destinazione è temporaneamente inattivo, **il messaggio rimane al sicuro nella coda** finché non torna online"*

A cosa serve anche: "distribuire il carico di lavoro" — se metti 5 istanze in ascolto sulla stessa coda, ogni messaggio va a **una sola** di esse (competing consumers).

Tecnologie: RabbitMQ con code dirette, AWS SQS, Azure Service Bus.

Tipo 2 — One-to-Many (Publish/Subscribe), con topic

Un producer pubblica un evento su un topic (o exchange). PIÙ consumer diversi possono sottoscrivere a quel topic. Ogni subscriber riceve UNA COPIA dell'evento e reagisce in modo indipendente.

"Questa comunicazione è il cuore del pattern publish/subscribe e il meccanismo alla base delle architetture event-driven."

★ L'aspetto cruciale — il disaccoppiamento:

"Il producer non sa né si preoccupa di quali servizi siano iscritti. Allo stesso modo, il consumer non ha bisogno di conoscere il producer. Si preoccupano solo degli eventi a cui sono interessati."

"È possibile aggiungere nuovi servizi subscriber per reagire a eventi esistenti, senza modificare affatto il servizio producer."

La retention degli eventi:

"Questi eventi possono anche essere conservati nel topic per un certo periodo, consentendo a nuovi subscriber di elaborare eventi storici, o ai servizi di riprodurre (replay) gli eventi."

Questa è una caratteristica distintiva di **Kafka** rispetto a RabbitMQ: il log degli eventi è persistente e rigiocabile.

Tecnologie: Apache Kafka, RabbitMQ con exchange e topic, AWS SNS, Azure Event Grid / Event Hubs.

★ Il confronto

	One-to-One (Queue)	One-to-Many (Topic)
Destinatari	Uno solo elabora il messaggio	Tutti i subscriber ricevono una copia
Semantica	Comando ("fai questo")	Evento ("questo è successo")
Dopo il consumo	Il messaggio viene rimosso	Ogni subscriber ha la sua copia
Aggiungere destinatari	Distribuisce il carico	Aggiunge comportamento
Esempio	"Crea l'ordine 123"	"L'ordine 123 è stato creato"

La distinzione semantica è importante: un **comando** esprime un'intenzione diretta a uno specifico destinatario ("*CreateOrder*", imperativo). Un **evento** è una constatazione di fatto ("*OrderCreated*", passato) di cui non sai chi si interesserà. Il primo mantiene un po' di accoppiamento; il secondo lo elimina quasi del tutto.

Combinare gli stili

*"Un'applicazione a microservizi nel mondo reale **utilizzerà quasi sempre una combinazione di questi stili**. Non è necessario sceglierne solo uno."*

Stile	Quando
Sincrono	"Interazioni client-servizio che richiedono risposta immediata — recuperare i dettagli del prodotto, validare l'input"
Async 1-a-1 (code)	"Quando un servizio deve inviare un comando o delegare un compito potenzialmente lungo e la risposta immediata non è critica "
Async 1-a-molti (pub/sub)	" Ampiamente , per implementare l'architettura event-driven"
<i>"La chiave è scegliere lo stile di comunicazione in base ai requisiti specifici dell'interazione: accoppiamento, latenza, affidabilità e coerenza."</i>	

Event-Driven Architecture and Publish/Subscribe Pattern

(Questo è il contenuto che avevamo già incontrato — ora completo e nel suo contesto.)

Definizione: Event-Driven Architecture

Un paradigma architetturale in cui il flusso del sistema è determinato dagli eventi.

Definizione di evento: *"un accadimento significativo, un cambiamento di stato, o una notifica che indica che qualcosa è accaduto all'interno di un servizio o del sistema."*

Esempi nel nostro dominio: OrderPlacedEvent, ProductPriceChangedEvent, UserRegisteredEvent, InventoryUpdatedEvent.

Il meccanismo:

"Nell'EDA, i servizi non effettuano chiamate sincrone dirette per comandare altri servizi. Invece, un servizio esegue un'azione e poi pubblica un evento per indicare che questa azione si è verificata. Altri servizi interessati possono sottoscriverlo."

"Questo promuove un accoppiamento estremamente debole, perché i servizi non hanno bisogno di conoscersi direttamente — solo gli eventi che producono o consumano."

Il pattern Publish/Subscribe — i 3 componenti

1. Publisher (editore)

"I servizi che creano e inviano eventi. La chiave è che i publisher inviano gli eventi a un canale centrale — il topic o event bus — senza sapere chi sono i subscriber, o persino se ci sono subscriber. Si limitano ad annunciare che è successo qualcosa."

2. Subscriber (iscritto)

"Servizi interessati a determinati tipi di eventi. Registrano il loro interesse iscrivendosi a uno specifico topic. Quando un evento viene pubblicato, ne ricevono una copia, lo consumano ed elaborano."

Esempio: Notification e Inventory possono entrambi sottoscrivere OrderPlaced.

3. Message Broker / Event Bus

"Il componente infrastrutturale intermedio che gestisce il topic e si occupa della consegna dagli editori a tutti gli iscritti interessati. Disaccoppia gli editori dagli iscritti."

★ Il modello fanout

"In questo modello, qualsiasi evento pubblicato su un topic viene ricevuto da TUTTI i subscriber attivi. Questo è noto anche come modello fanout, in cui un evento viene distribuito a più consumatori in parallelo."

I 4 vantaggi

1. Disaccoppiamento — "il vantaggio più significativo"

*"Publisher e subscriber **non dipendono direttamente gli uni dagli altri**. Dipendono solo dal formato dell'evento e dall'event bus. È possibile **modificare, aggiornare o distribuire un subscriber senza impattare il publisher**, e aggiungere nuovi subscriber senza che il publisher se ne accorga."*

2. Scalabilità

*"Publisher, subscriber e il broker stesso **possono essere scalati in modo indipendente**. Se si hanno molti eventi di un certo tipo, si può aumentare il numero di subscriber per elaborarli in parallelo."*

*Esempio citato: **KEDA** (Kubernetes Event-Driven Autoscaling) — "può scalare automaticamente i microservizi **in base al carico del message broker**".*

3. Resilienza

*"Se un subscriber **non è temporaneamente disponibile**, il broker può conservare gli eventi. Una volta tornato online, può elaborare gli eventi in coda. In questo modo si evita la perdita di operazioni a causa di guasti temporanei a valle."*

4. Reattività

*"Il servizio che pubblica un evento **può farlo rapidamente e passare ad altro, senza aspettare che l'evento venga elaborato da tutti i subscriber**. L'operazione iniziale risulta molto più veloce per l'utente."*

L'esempio riassuntivo

*"Quando viene effettuato un ordine, **Ordering pubblica l'evento OrderPlaced**. Poi **Inventory, Notification e Shipping** possono tutti sottoscrivere questo evento ed eseguire le rispettive attività in modo asincrono e parallelo. Ordering non ha bisogno di effettuare tre chiamate sincrone separate e attenderle."*

Confronta con la catena della Lezione 8: **6 chiamate sequenziali, 600ms, 94% di disponibilità → 1 pubblicazione, risposta immediata, resilienza ai guasti dei consumer.**

★ Le 5 sfide

1. Il message broker è un componente critico

*"Può diventare **un single point of failure** se non distribuito in configurazione altamente disponibile e resiliente. **Anche prestazioni e scalabilità sono fondamentali.**"*

2. Debug e tracing dei flussi distribuiti

*"Quando un'operazione è avviata da un evento e coinvolge più servizi che reagiscono indipendentemente, **tracciare il flusso end-to-end e debuggare i problemi può essere più complesso delle catene sincrone**. Un buon sistema di distributed tracing e logging è essenziale."*

Nota il trade-off: nella catena sincrona avevi almeno **uno stack di chiamate**. Nell'EDA non c'è.

3. Eventual Consistency

"Poiché i servizi elaborano gli eventi in modo asincrono, i dati tra microservizi diventeranno coerenti ALLA FINE, non immediatamente dopo la pubblicazione. Il sistema e l'esperienza utente devono essere progettati per gestire questa situazione."

4. Garanzia dell'ordine dei messaggi

"Alcuni broker garantiscono l'ordine all'interno di una partizione o per uno specifico tipo di coda, ma l'ordine globale tra tutti i consumer spesso NON è garantito. Se l'ordine rigoroso è fondamentale, occorre una progettazione accurata."

5. Consumer idempotenti ★

"A causa di problemi di rete o di meccanismi di retry del broker, un consumer potrebbe ricevere lo stesso evento più di una volta. I consumer devono essere idempotenti: elaborare lo stesso evento più volte deve avere lo stesso effetto di elaborarlo una volta sola."

Perché succede tecnicamente: la maggior parte dei broker garantisce **at-least-once delivery** (almeno una volta). Se il consumer elabora il messaggio ma il suo *ack* si perde per un problema di rete, **il broker rispedisce**. Garantire *exactly-once* end-to-end è molto costoso; è più economico **rendere idempotente il consumer** (es. tracciando gli event Id già processati).

Design: Microservices Architecture w/ EDA and Publish/Subscribe

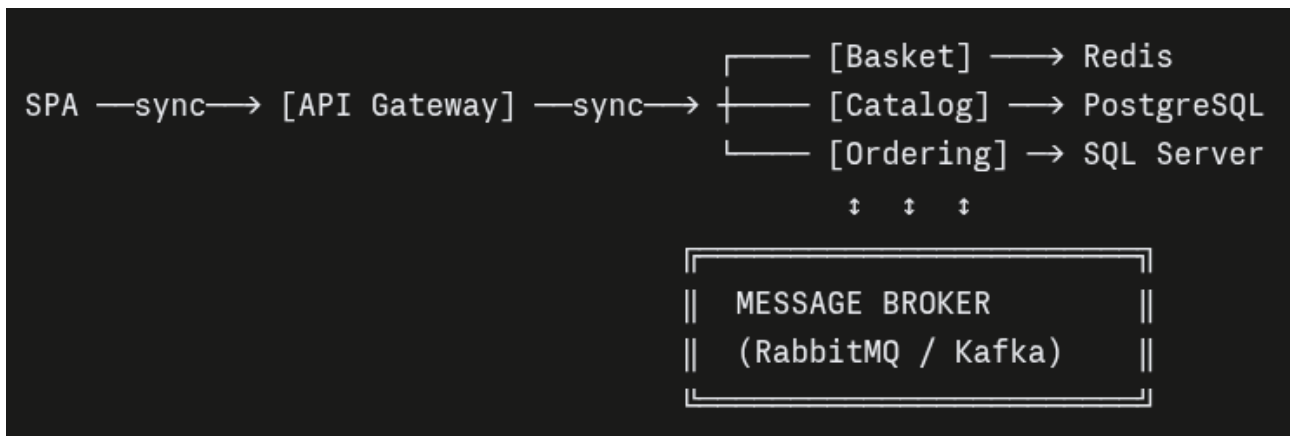
Toolkit aggiornato

Nuovi requisiti non funzionali:

1. **Elevato disaccoppiamento tra servizi**, soprattutto per processi multi-step o notifiche
2. **Migliorata resilienza e fault tolerance** — *"il sistema deve gestire con grazia la temporanea indisponibilità dei servizi downstream"*
3. **Scalabilità dell'elaborazione degli eventi**, grazie alla possibilità di avere più consumer
4. ★ **Progettare esplicitamente per la eventual consistency**, dove opportuno

Nuovi pattern: Event-Driven Architecture e Publish/Subscribe.

Il design

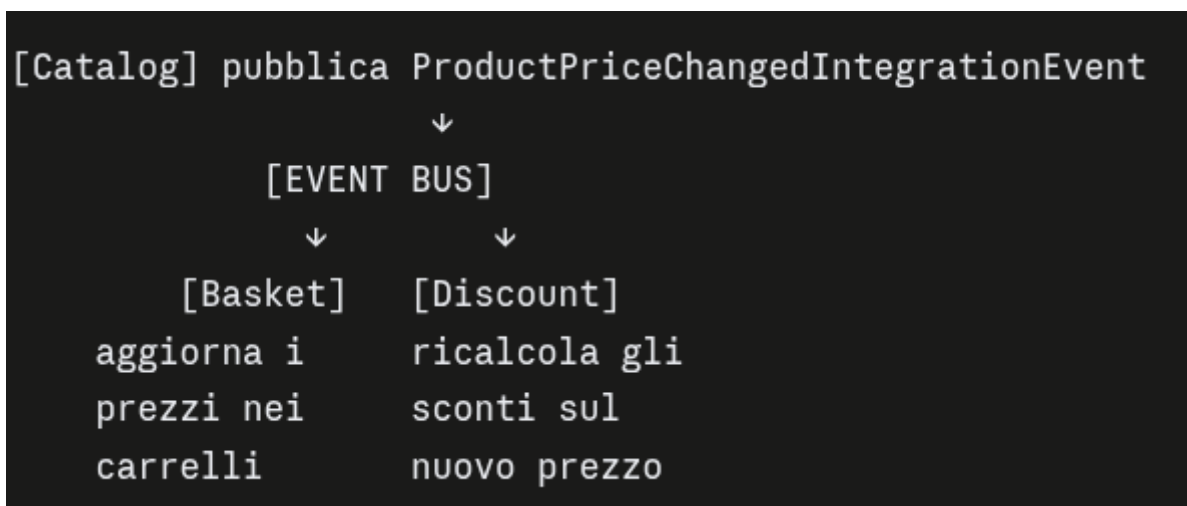


Cosa NON cambia: *"La nostra applicazione client interagisce ancora con l'API Gateway usando chiamate sincrone."*

Cosa cambia: *"Stiamo introducendo un nuovo componente: il message broker / event bus. È qui che gli eventi vengono pubblicati e da qui vengono consumati."*

★ I due esempi di flusso

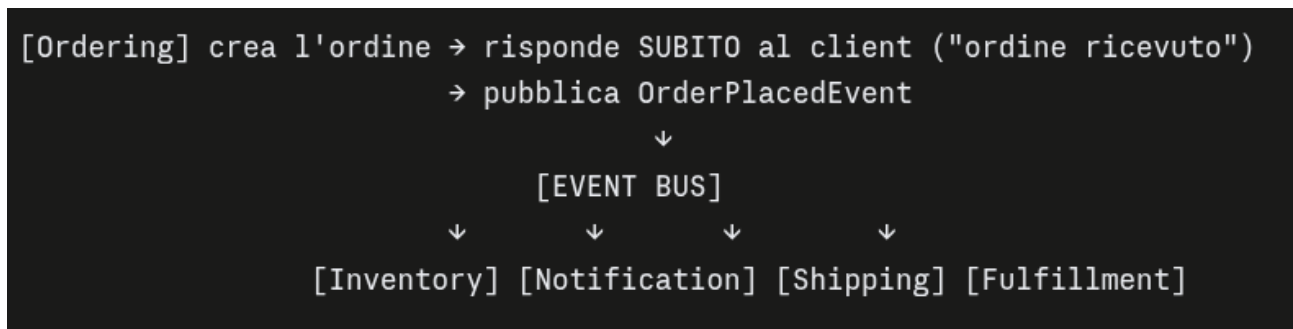
Esempio A — Cambio di prezzo



*"Quando un prezzo viene aggiornato nel Catalog, **invece di cercare di comunicare sincronamente con tutti gli altri servizi**, pubblica semplicemente l'evento sull'event bus. **Il servizio Catalog non ha bisogno di sapere chi è in ascolto.**"*

*"Basket e forse Discount si sottoscrivono a questo evento **per aggiornare i prezzi nei carrelli attivi**. E se gli sconti sono percentuali, **si possono applicare in base all'ultimo prezzo.**"*

Esempio B — Ordine effettuato



"Il lavoro di Ordering per quella richiesta immediata è ora concluso, e può rispondere rapidamente al cliente."

"Il servizio Ordering non chiama direttamente né dipende da Inventory o Notification per l'accettazione iniziale dell'ordine."

Scelte tecnologiche

Opzione	Caratteristiche
Apache Kafka	"Molto popolare, soprattutto per event streaming ad alto throughput e per sistemi che richiedono replay degli eventi o stream processing . Progettato per scalabilità orizzontale e fault tolerance "
RabbitMQ	" Molto robusto e flessibile . Broker tradizionale che supporta vari protocolli (AMQP) e pattern che includono code, publish/subscribe e routing "
Cloud gestiti	AWS: SNS (topic), SQS (code), EventBridge (routing avanzato). Azure: Service Bus, Event Grid. Google: Pub/Sub
	"La scelta dipende da: throughput richiesto, esigenze di persistenza dei messaggi, complessità del routing , e se si preferisce un servizio gestito o self-hosted ."

I 5 svantaggi

1. **Aumento della complessità del sistema** — "comprendere il flusso end-to-end è più difficile. Non c'è uno stack di chiamate diretto"
2. **Difficile da debuggare e tracciare** — "richiede un buon strumento di osservabilità"
3. **Eventual consistency** — "i dati tra servizi **non sono immediatamente coerenti**"
4. **Il broker è critico** — "potenziale single point of failure o collo di bottiglia; deve essere altamente disponibile e scalabile"
5. **Nessuna garanzia sull'ordine dei messaggi** — "e tutte le operazioni di consumo devono essere il più possibile idempotenti"

Code: Microservices Architecture w/ EDA and Publish/Subscribe

Cartella 9-eshop-microservices-async-rabbitmq. Broker: **RabbitMQ**. Libreria: **MassTransit**.

Definizione: MassTransit — un framework .NET per la messaggistica distribuita che astrae il broker sottostante (RabbitMQ, Azure Service Bus, Amazon SQS). Fornisce le astrazioni `IPublishEndpoint` per pubblicare e `IConsumer<T>` per consumare.

★ Domain Events vs Integration Events

Questa distinzione è importante e viene introdotta qui:

	Domain Event (giallo nelle slide)	Integration Event (arancione)
Ambito	Interno a un singolo microservizio	Tra microservizi
Dove viaggia	In-process (mediator, in-memory)	Sull'event bus (RabbitMQ)
Esempio	ProductPriceChangedDomainEvent dentro Catalog	ProductPriceChangedIntegrationEvent sul bus

"I microservizi Catalog sollevano internamente l'evento di dominio relativo alla modifica del prezzo, e questo evento di dominio pubblica poi l'evento di integrazione sull'event bus."

"In questa lezione ci interesseremo agli eventi di integrazione, perché ci concentriamo sui sistemi distribuiti."

I due casi d'uso implementati

```
CASO 1 – Cambio prezzo prodotto
  Client → PUT /products/1 → [Catalog]
                                ↳ pubblica ProductPriceChangedIntegrationEvent →
RabbitMQ
                                ↳ aggiorna PostgreSQL
                                ↓
                                [Basket] consuma → aggiorna i prezzi degli item → Redis

CASO 2 – Checkout del carrello
  Client → POST /basket/checkout → [Basket]
                                ↳ pubblica BasketCheckoutIntegrationEvent →
RabbitMQ
                                ↳ cancella il carrello
                                ↓
                                [Ordering] consuma → crea l'ordine → SQL Server
```

Nota il cambiamento rispetto alla Lezione 7/8: prima Basket chiamava Ordering **sincronamente** durante il checkout. **Ora pubblica un evento e ha finito.** Se Ordering è down, l'evento resta in coda e viene processato quando torna su — l'utente non se ne accorge.

1. AppHost — aggiungere RabbitMQ

// BACKING SERVICES

```
var messaging = builder.AddRabbitMQ("messaging")
                        .WithManagementPlugin(); // ← UI di gestione
```

// MICROSERVIZI — ognuno referencia il broker

```
builder.AddProject<Catalog>("catalog").WithReference(messaging);
builder.AddProject<Basket>("basket").WithReference(messaging);
builder.AddProject<Ordering>("ordering").WithReference(messaging);
```

*"L'unica differenza fondamentale è che ora aggiungiamo RabbitMQ tra i backing services. Includiamo il **management plugin**, così possiamo accedere all'interfaccia di gestione per vedere e tracciare gli eventi."*

*"Ogni microservizio ora riceve un riferimento a RabbitMQ. Questo fornisce i dettagli della **connection string**, in modo da poter pubblicare e consumare eventi."*

*"La struttura delle cartelle rimane invariata, poiché stiamo solo aggiungendo il **message broker** al progetto."*

2. Il lato PUBLISH — Catalog/Services/ProductService.cs

```
public async Task UpdateProductAsync(Product product)
{
    // 1. Controlla se il prezzo è cambiato
    if (existingProduct.Price != product.Price)
    {
        // 2. Crea l'evento di integrazione
        var integrationEvent = new ProductPriceChangedIntegrationEvent
        {
            ProductId = product.Id,
            ProductName = product.Name,
            Description = product.Description,
            Price = product.Price,    // ← il campo che conta
            ImageUrl = product.ImageUrl
        };

        // 3. PUBBLICA sull'event bus
        await _publishEndpoint.Publish(integrationEvent);
    }

    // 4. Continua con l'aggiornamento del database
    await _dbContext.SaveChangesAsync();
}
```

"Pubblichiamo l'evento al message broker RabbitMQ, che verrà consumato dai microservizi Basket per aggiornare i prezzi del carrello. Dopo la pubblicazione, l'operazione continua per l'aggiornamento degli altri campi e del database."

⚠ **Osservazione importante:** nota che qui si **pubblica prima** e si **salva il database dopo**. Se il salvataggio fallisse, avresti pubblicato un evento per un cambiamento mai avvenuto. Questo è precisamente il problema che il **pattern Outbox** risolve — e infatti nel repository esiste la cartella `11-eshop-microservices-outbox`.

3. Il lato CONSUME —

Basket/EventHandlers/ProductPriceChangedEventHandler.cs

```
public class ProductPriceChangedEventHandler
{
    : IConsumer<ProductPriceChangedIntegrationEvent> // ← MassTransit
{
    public async Task Consume(ConsumeContext<ProductPriceChangedIntegrationEvent> context)
    {
        // trova gli item del carrello con quel prodotto
        // aggiorna il prezzo
        // riscrive il carrello in Redis
        await _basketService.UpdateItemPrice(
            context.Message.ProductId,
            context.Message.Price);
    }
}
```

*"La classe eredita dall'interfaccia **IConsumer<T>**, specificando il tipo di evento. Tramite questa interfaccia, **MassTransit** ascolta gli eventi di **RabbitMQ** da un servizio in background e li consuma quando arrivano."*

Il metodo `Consume` trova il prodotto negli item del carrello, imposta il prezzo, e aggiorna la cache.

4. Il secondo caso d'uso — Checkout

Basket/Services/BasketService.cs, metodo CheckoutBasket:

1. Recupera il carrello esistente con il prezzo totale
2. Imposta il totale nell'oggetto `BasketCheckout`
3. Crea il `BasketCheckoutIntegrationEvent`
4. **Pubblica** l'evento su RabbitMQ
5. **Cancella il carrello**

Ordering/EventHandlers/BasketCheckoutEventHandler.cs:

- Consuma l'evento

- **Crea un nuovo ordine nel database SQL Server** usando i dati dell'evento in arrivo

5. Cosa si osserva eseguendo

Nuova risorsa: il container **RabbitMQ**, con un endpoint per la **management UI**.

Il graph view:

*"La webapp invia richieste al gateway, che le reindirizza ai microservizi interni. **Ma questa volta questi microservizi eseguono anche comunicazione asincrona tramite l'event bus RabbitMQ. Si può notare che RabbitMQ ha un riferimento per ogni microservizio, che può pubblicare e sottoscrivere.**"*

Accesso alla management UI: username e password (guest) si trovano nelle **variabili d'ambiente del container RabbitMQ** nella dashboard Aspire. Dentro la UI si vedono **exchange, code, connessioni, canali, stream**.

★ Il test end-to-end del cambio prezzo

Questa demo è molto efficace:

1. Nella webapp, si aggiunge al carrello una **"torcia a energia solare"** → il carrello mostra **totale 19,99**
2. Si apre `catalog.http` in Visual Studio e si invia:

```
PUT /products/1
```

```
{ "name": "...", "price": 5.99, ... } ← prezzo cambiato da 19.99 a 5.99
```

3. **Breakpoint 1** — `ProductService.UpdateProductAsync`: si vede che il prezzo è cambiato da 19 a 5 → si crea e **pubblica** l'evento
4. **F5** → **Breakpoint 2** — `ProductPriceChangedEventHandler.Consume` **nel microservizio Basket**: si trova il `ProductId` e **si aggiorna il prezzo dell'item**
5. **F5** → operazione completata (204 No Content)
6. **Nella management UI di RabbitMQ**: si vede **un hit** sull'exchange `ProductPriceChangedIntegrationEvent`, con la coda del consumer `Basket`
7. **Si ricarica il carrello nella webapp** → **il prezzo totale è aggiornato a 5,99** ✓

*"Quando aggiorniamo il prodotto nel Catalog, **il prezzo dell'item nel carrello viene aggiornato di conseguenza. E tutte queste comunicazioni vengono eseguite in modo asincrono, usando il pattern publish/subscribe.**"*

La traccia in Aspire:

```

PUT /products/1 → [Catalog]
    | SELECT su PostgreSQL
    | PUBLISH ProductPriceChangedIntegrationEvent ← operazione di
publish
    | UPDATE su PostgreSQL
    | ↓ (asincrono)
[Basket] CONSUME ProductPriceChangedEventHandler
    | ↳ aggiorna Redis

```

Il test del checkout

1. Si clicca Checkout nella webapp, si compila il form
2. **Breakpoint** in `BasketService.CheckoutBasket` → calcola e crea l'evento → **pubblica**
3. **F5** → **Breakpoint** nel Consume di **Ordering** → **crea l'ordine in SQL Server**
4. Il checkout risulta completato, l'ordine compare nella lista

La traccia:

```

POST /basket-service/basket/checkout → [Gateway] → [Basket]
    | operazioni interne
    | ↳ PUBLISH BasketCheckoutEvent
RabbitMQ
    |
    | ↓
    | [Ordering] CONSUME
    | ↳ INSERT in SQL Server

```

"Abbiamo dimostrato con successo la comunicazione asincrona con RabbitMQ, seguendo il modello event-driven e publish/subscribe."

The Scale Cube & Stateful Application Horizontal Scaling

Questa sezione **chiude il cerchio con le Lezioni 4 e 6** e apre il tema della Lezione 10.

Ripasso dello Scale Cube — e dove siamo

Asse	Nome	Dove l'abbiamo applicato nel corso
X	Horizontal scaling / cloning	✓ Lezione 4 — <code>WithReplicas(3)</code> sull'API Service
Y	Functional decomposition	✓ Lezioni 5-6 — decomposizione in Catalog, Basket, Ordering
Z	Data partitioning / sharding	✗ Non ancora

"Ciò significa che d'ora in poi ci concentreremo sullo scaling dell'asse Z: partizionamento dei dati o sharding. Lo esploreremo nelle prossime lezioni sulla gestione dei dati."

Esempio di asse Z dato: un'istanza gestisce i clienti con nome da A a M, un'altra da N a Z.

★ La distinzione cruciale: stateless vs stateful

"Quando si parla di scaling orizzontale (asse X), è necessario fare una distinzione cruciale in base al fatto che l'istanza detenga o meno lo STATO."

Applicazioni STATELESS

"La logica dei nostri microservizi è progettata per essere stateless: non memorizza alcun dato specifico del client tra una richiesta e l'altra nella propria memoria. Qualsiasi richiesta può essere gestita da qualsiasi istanza disponibile."

"Questo rende relativamente facile lo scaling orizzontale: basta aggiungere altre istanze identiche e un load balancer. Gli orchestratori come Kubernetes sono eccellenti per gestire e scalare applicazioni stateless."

Applicazioni STATEFUL

"I database sono il primo esempio di servizi stateful. Memorizzano dati che persistono e cambiano nel tempo."

Le domande difficili:

"Se si cerca semplicemente di eseguire più copie identiche di un database relazionale tradizionale:"

- *"Come si garantisce che una scrittura su una copia si rifletta su tutte le altre?"*
- *"Come si gestiscono i conflitti, se due utenti cercano di aggiornare lo stesso dato su istanze diverse contemporaneamente?"*

"È qui che la scalabilità orizzontale diventa molto più impegnativa."

Il nodo del problema

"I nostri microservizi sono per lo più stateless e possono scalare, ma i loro database dedicati sono stateful. La scalabilità verticale di questi database ha limiti e costi elevati. E quando cerchiamo di replicarli, si creano grandi problemi di coerenza."

Problem: Databases Create Bottlenecks When Scaling

L'audit di scalabilità componente per componente

La lezione fa un esercizio molto utile: passa in rassegna **ogni componente** dell'architettura e chiede "può scalare orizzontalmente?"

Componente	Stateless?	Può scalare orizzontalmente?	Perché
Client apps (SPA,	✓	✓ Sì	"Servite da infrastrutture scalabili

Componente	Stateless?	Può scalare orizzontalmente?	Perché
mobile)			come CDN o hosting web scalabile"
API Gateway / BFF	✓	✓ Sì	"Tipicamente progettati per essere stateless → più istanze dietro un load balancer "
Microservizi (Catalog, Basket, Ordering)	✓	✓ Sì	" Progettati appositamente per essere il più possibile stateless "
Message Broker (RabbitMQ/Kafka)	—	✓ Sì	"Costruiti per ambienti distribuiti, con potenti caratteristiche di clustering "
Database (PostgreSQL, Redis, SQL Server)	✗ STATEFUL	✗ NO ★	—

"Ora la domanda cruciale: **che ne è dei database di cui ogni microservizio è proprietario?** Catalog DB (PostgreSQL), Basket DB (Redis), Ordering DB (SQL Server)."

"Quando cerchiamo di gestire milioni di richieste, la risposta è **NO**, perché i database sono applicazioni stateful e presentano sfide significative rispetto ai servizi applicativi stateless."

Nota quanto è elegante questo ragionamento: tutto ciò che abbiamo costruito nelle lezioni 4-9 scala. Il collo di bottiglia si è spostato progressivamente, e ora è arrivato all'ultimo strato: i dati.

Il problema definito

"Databases become bottlenecks when scaling."

"Mentre è relativamente facile aumentare le istanze dei nostri microservizi stateless, scalare i database che contengono i dati reali è un problema molto più difficile."

I 3 problemi specifici

1. I database sono intrinsecamente stateful

"Il loro scopo è memorizzare e gestire lo stato in modo affidabile e coerente. Questa statefulness rende molto complesso il semplice scaling orizzontale."

2. Lo scaling verticale ha limiti

"Per un database relazionale come PostgreSQL o SQL Server, il primo approccio è spesso lo scaling verticale — comprare server più grandi e potenti. Ma ci sono limiti fisici e costi esponenziali. Non è possibile scalare verticalmente per sempre."

Esattamente il limite identificato nella **Lezione 4**, ora applicato al livello dati.

3. Mantenere la coerenza su più istanze ★

"Se si cerca di scalare orizzontalmente un database relazionale tradizionale semplicemente avendo più copie in lettura e scrittura, garantire la coerenza dei dati, gestire i conflitti di scrittura e gestire la replica diventa incredibilmente complesso."

La conclusione

"Mentre i nostri servizi applicativi possono scalare, i database non riescono a tenere il passo con la domanda di letture e scritture. Questi database diventano il collo di bottiglia e limitano la scalabilità complessiva dell'intera architettura."

La roadmap della soluzione:

*"Dobbiamo apprendere **strategie di gestione dei dati e tecnologie di database** in grado di gestire carichi elevati di lettura/scrittura e **scalare più facilmente in orizzontale**. Nelle prossime sezioni esploreremo:"*

- **diversi tipi di database: SQL e NoSQL**
 - **pattern di partizionamento e sharding dei dati**
 - **funzionalità avanzate di replica e clustering**
-